groove-marketing-web — Setup & Production Release Runbook

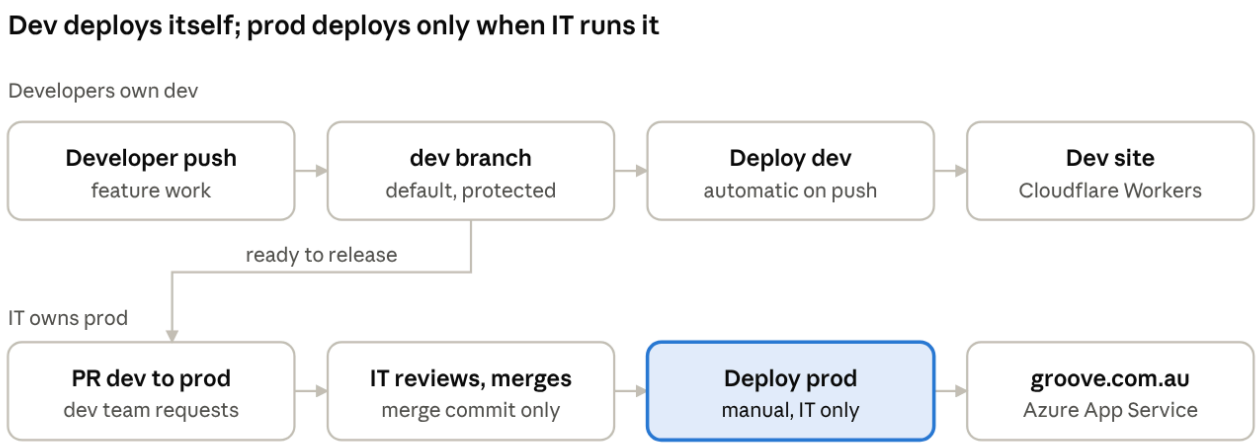
Oct 13, 2026 · @IT Operations

Overview

Developers own the `dev` branch, which deploys automatically to Cloudflare Workers. IT owns `prod`, which deploys to Azure App Service only when IT runs the Deploy prod workflow by hand. Merging into `prod` never deploys on its own.

Item	Dev	Prod
Branch	`dev` (default branch)	`prod`
Who pushes	Developers	IT only, via PR merge from `dev`
Hosting	Cloudflare Workers (static assets)	Azure App Service `prod-groove-marketingweb-appsvc-aue` (Australia East)
URL	https://groove-marketing-web-dev.development-312.workers.dev/	groove.com.au (after go-live)
Deploy trigger	Automatic on every push to `dev`	Manual: Actions → Deploy prod → Run workflow
Workflow file	`.github/workflows/deploy-dev.yml`	`.github/workflows/deploy-prod.yml`
Auth to cloud	Cloudflare API token (env secret)	Azure OIDC with a user-assigned managed identity (no stored password)
GitHub environment	`dev`	`prod` (restricted to PROD_DEPLOYERS)
Search engines	Blocked (noindex, robots disallow)	Indexed

Repository: `JRC-Software/groove-marketing-web`. The site is static (`index.html` , `css/` , `js/` , `assets/`) with no build step; prod serves it through a small Node host (`server.js`) that also answers `/healthz` .



release flow · dev to prod

The only gate between a developer's push and the live site is IT running Deploy prod after merging the release PR.

Prerequisites

Have these accounts and access in place before starting Part 1.

- [ ] GitHub: admin on `JRC-Software/groove-marketing-web`; org plan Team or higher (needed for environment protection rules on a private repo)
- [ ] Azure: Owner (or Contributor + User Access Administrator) on the subscription, so you can create the managed identity and its role assignment
- [ ] Cloudflare: access to Development@jrcsoftware.com's Account (Workers) and to the zone for groove.com.au (DNS)
- [ ] GoDaddy: domain registrar login, nameservers already pointing at Cloudflare
- [ ] Tools: Azure Cloud Shell (Bash) in the portal for the `infra/` and `scripts/` files, since `.sh` scripts do not run in Windows cmd; Git and a browser for GitHub

Azure resources used (prod only, Australia East):

Resource	Name	Notes
Resource group	`prod-groove-marketingweb-rg-aue`	Created
App Service plan	`prod-groove-marketingweb-asp-aue`	Linux; F1 for testing, B1 needed for go-live
Log Analytics	`prod-groove-marketingweb-log-aue`	App logs

Resource	Name	Notes
Web app	`prod-groove-marketingweb-appsvc-aue`	Node 24, start command `node server.js`
Managed identity	`prod-groove-marketingweb-id-aue`	Used by GitHub Actions via OIDC
Role assignment	Website Contributor	On the web app only

Step 1 — GitHub repository, branches and environments

Done once, in the GitHub web UI. Status: `dev` side complete (2026-10-07); `prod` side pending.

1. **Create the repo** `JRC-Software/groove-marketing-web` (private). Push the site files and the `.github/`, `infra/` and `scripts/` folders to a branch named `dev`.
2. **Create the `prod` branch** from `dev`: Code → branch dropdown → type `prod` → Create branch from dev.
3. **Set the default branch** to `dev`: Settings → General → Default branch. Developers then clone and open PRs against `dev` by default.
4. **Protect `dev`**: Settings → Rules → Rulesets → New branch ruleset → target `dev` → enable *Restrict deletions* and *Block force pushes*.
5. **Protect `prod`**: new ruleset → target `prod` → enable:
   - Restrict deletions, Block force pushes
   - Require a pull request before merging (1 approval)
   - Require status checks to pass → add `ci`
   - Bypass list: none. Only IT should be able to merge, so give the dev team Write access and IT Maintain/Admin, or restrict merges via the ruleset.
6. **Create the `dev` environment**: Settings → Environments → New environment → `dev` → Deployment branches: *Selected branches* → `dev`. Add environment secrets:
   - `CLOUDFLARE_API_TOKEN`
   - `CLOUDFLARE_ACCOUNT_ID`
7. **Create the `prod` environment**: New environment → `prod` →
   - Deployment branches: *Selected branches* → `prod` only
   - Required reviewers: the IT team (optional second gate)
   - Prevent self-review: off if IT is a single person

- Secrets: `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID` (values from Step 2)
- Variable: `PROD_DEPLOYERS` = comma-separated GitHub logins allowed to run Deploy prod (IT only)

8. **Teams**: create a dev team (Write) and an IT team (Maintain or Admin) under the org, and add them to the repo under Settings → Collaborators and teams.

Step 2 — Azure (prod hosting and OIDC identity)

Run in Azure Cloud Shell (Bash) from a clone of the repo. The Bicep file `infra/main.bicep` builds everything in two stages.

1. **Open Cloud Shell** in the Azure portal and clone the repo:

```
git clone https://github.com/JRC-Software/groove-marketing-web.git
cd groove-marketing-web
az account set --subscription "<subscription name or id>"
```

2. **Confirm the resource group** exists (it does): `az group show -n prod-groove-marketingweb-rg-aue`.
3. **Stage `shared`** — creates the App Service plan and Log Analytics:

```
az deployment group create -g prod-groove-marketingweb-rg-aue \
  --template-file infra/main.bicep --parameters stage=shared
```

4. **Get the GitHub IDs** for the OIDC subject. Open `https://api.github.com/repos/JRC-Software/groove-marketing-web` (signed in) and note `id` (repo ID) and `owner.id` (owner ID).
5. **Stage `prod`** — creates the web app, managed identity, federated credential and the Website Contributor role on the app. Pass the two IDs from step 4 (parameter names as defined in `main.bicep` / `scripts/provision.sh`):

```
az deployment group create -g prod-groove-marketingweb-rg-aue \
  --template-file infra/main.bicep --parameters stage=prod \
  githubOwnerId=<ownerId> githubRepoId=<repoId>
```

   The federated subject must read exactly `repo:JRC-Software@<ownerId>/groove-marketing-web@<repoId>:environment:prod`. Check it under Managed Identities → `prod-groove-marketingweb-id-aue` → Federated credentials.
6. **Collect the three values** for GitHub `prod` secrets:

```
az identity show -g prod-groove-marketingweb-rg-aue -n prod-groove-
marketingweb-id-aue --query clientId -o tsv
az account show --query tenantId -o tsv
az account show --query id -o tsv
```

Paste them as `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID` in the `prod` environment (Step 1.7).

7. **Before go-live**, scale the plan to B1 (F1 has no custom domains or SSL; about AUD 20/month):

```
az appservice plan update -g prod-groove-marketingweb-rg-aue -n prod-
groove-marketingweb-asp-aue --sku B1
```

No passwords or publish profiles are stored in GitHub: the workflow exchanges a short-lived GitHub token for an Azure token, and only jobs running in the `prod` environment can do it.

Step 3 — GitHub Actions workflows

The three workflows are already committed on `dev`; they reach `prod` with the first release PR.

Workflow	Trigger	What it does
`ci.yml`	Any PR into `dev` or `prod`	Job `ci`: packages the site and checks links. Required check on `prod`.
`deploy-dev.yml`	Push to `dev`	Packages the site, adds noindex headers, robots disallow and `build-info.json`, deploys to Cloudflare Workers with `wrangler-action`. Uses the `dev` environment.
`deploy-prod.yml`	Manual only (`workflow_dispatch`, required `release` input)	Build job (no Azure token) packages the site. Deploy job runs in the `prod` environment, checks the runner is in `PROD_DEPLOYERS`, logs in to Azure via OIDC, deploys to the web app, then calls `/healthz` and confirms it reports the deployed commit.

Verify the setup once:

1. Push a small change to `dev` → Actions shows *Deploy dev* green → the workers.dev URL shows the change.
2. Open a test PR into `prod` → the `ci` check runs and must pass before merge is allowed.
3. After the first release (Part 3), confirm *Deploy prod* fails for a user not in `PROD_DEPLOYERS`.

Step 4 — Domain: GoDaddy → Cloudflare → Azure (go-live only)

Do this once, after the first successful prod deploy and after scaling to B1. GoDaddy stays the registrar; all DNS records live in Cloudflare.

1. **Check nameservers**: in GoDaddy → Domain → DNS → Nameservers, confirm they are the two Cloudflare nameservers shown in Cloudflare → groove.com.au → Overview.
2. **Get the Azure values**: Azure portal → `prod-groove-marketingweb-appsvc-aue` → Custom domains → Add custom domain. Note the **IP address** and the **Custom Domain Verification ID**.
3. **Add DNS records in Cloudflare** (DNS → Records), all with proxy **off (grey cloud)**:

Type	Name	Content
A	`@`	App Service IP from step 2
TXT	`asuid`	Custom Domain Verification ID
CNAME	`www`	`prod-groove-marketingweb-appsvc-aue.azurewebsites.net`
TXT	`asuid.www`	Custom Domain Verification ID

4. **Bind the domains** in Azure: Custom domains → add `groove.com.au`, then `www.groove.com.au` (or run `scripts/bind-custom-domains.sh` in Cloud Shell). Validation passes once DNS has propagated (usually minutes).
5. **Add free SSL**: for each domain choose *App Service Managed Certificate* and binding type *SNI SSL*.
6. **Force HTTPS**: web app → Configuration (or TLS/SSL settings) → HTTPS Only = On.
7. **Test**: https://groove.com.au and https://www.groove.com.au both load with a valid certificate.

Keep the records grey-cloud: proxying through Cloudflare (orange) can block Azure managed-certificate renewal. If you later want Cloudflare proxying, switch SSL mode to Full (strict) and plan for certificate renewal first.

Developer workflow (dev environment)

The dev environment belongs to the developers; every push to `dev` goes live on the workers.dev URL within a few minutes.

1. Clone the repo; `dev` is checked out by default.
2. Work on a feature branch, open a PR into `dev`, wait for the `ci` check, merge (or push straight to `dev` if the team allows it).
3. *Deploy dev* runs automatically. Check https://groove-marketing-web-dev.development-312.workers.dev/ and `/build-info.json` to confirm the commit.
4. When a set of changes is ready for release, the dev team opens a **PR from `dev` into `prod`** with a title like `Release 2026-10-20` and a short list of what changed. This PR is the request for IT to deploy.

Developers never push to `prod` and cannot run *Deploy prod*.

Production release (IT, every release)

A release is three actions by IT: approve the dev→prod PR, merge it with a merge commit, then run *Deploy prod* manually. Allow about 15 minutes.

A. Before merging — confirm the release is approved

1. Get written sign-off from the dev team lead (in the PR, a comment like "Approved for prod").
2. Open the PR: Pull requests → the `dev` → `prod` PR. Base must be `prod`, compare must be `dev`.
3. Check the **Files changed** tab: only expected site files; no edits to `.github/workflows/` or `infra/` unless agreed. Workflow changes affect how prod deploys, so review them line by line.
4. Check the `ci` check is green.
5. Spot-check the dev site: https://groove-marketing-web-dev.development-312.workers.dev/ — home page, navigation, pitch form (mailto), images, mobile view.

B. Merge the PR

6. Approve the PR (Review changes → Approve).
7. Merge with **Create a merge commit** (not squash or rebase), so `dev` and `prod` share history and the next PR stays clean.
8. Merging does **not** deploy. Nothing changes on groove.com.au yet.

C. Deploy to production

9. Go to Actions → **Deploy prod** → **Run workflow**.

10. Branch: `prod`. Release: a label such as `2026-10-20` or `v1.4`. Click **Run workflow**.
11. If the `prod` environment has required reviewers, click **Review deployments** → tick `prod` → **Approve and deploy**.
12. Wait for both jobs (build, deploy) to go green. The deploy job's smoke test calls `/healthz` and fails the run if it does not report the new commit.

D. Verify and close out

13. Open https://groove.com.au (or the `azurewebsites.net` URL before go-live) in a private window; hard-refresh. Check the changed pages.
14. Check https://groove.com.au/healthz shows the commit SHA from the merge.
15. Post in the team channel: release label, commit SHA, time, and the PR link.

If anything fails: stop and use *Rollback* below. Do not push fixes to `prod` directly; fixes go through `dev` and a new PR.

Rollback, hotfixes and troubleshooting

Rollback (bad release is live):

1. Open the merged release PR → click **Revert**. GitHub opens a new PR into `prod` that undoes it.
2. Merge that revert PR (merge commit), then run **Deploy prod** again with release label e.g. `2026-10-20-rollback`.
3. Verify the site and `/healthz` show the previous state.
4. Tell the dev team. When they re-release the fixed changes, the next dev→prod PR must include a revert of the revert, or Git will treat the original changes as already undone.

Hotfix: same path as a normal release, just faster: fix on `dev` → check dev site → PR `dev` → `prod` → merge → Deploy prod. Never commit straight to `prod`.

Troubleshooting:

Symptom	Likely cause	Fix
Deploy prod fails at the deployer check	Your login is not in `PROD_DEPLOYERS`	Add it in Settings → Environments → prod → Variables
Azure login fails with `AADSTS70021` / no matching federated identity	Subject mismatch (wrong environment, repo renamed or IDs wrong)	Compare the federated credential subject with Step 2.5; run from branch `prod`

Symptom	Likely cause	Fix
Deploy job says branch not allowed	Workflow run on `dev`	Run workflow with branch `prod`
Smoke test fails on `/healthz`	App not started or old build still served	Web app → Log stream; restart the app; re-run the job
Merge button disabled on dev→prod PR	`ci` failing or approval missing	Ask devs to fix on `dev`; approve the PR
PR shows unrelated old commits	Earlier merge used squash/rebase	Always use merge commits; if needed merge `prod` back into `dev` once
Custom domain will not validate	TXT `asuid` missing or proxied record	Check records, grey cloud, wait for DNS
Deploy dev fails with auth error	Cloudflare token expired or revoked	Create a new token (Workers Scripts:Edit, Account Settings:Read) and update the `dev` secret

Checklists and open items

Setup status:

- [x] ~~Resource group `prod-groove-marketingweb-rg-aue` created~~
- [x] ~~Dev path: Cloudflare token, `dev` environment + secrets, `dev` protected and default, Deploy dev green~~
- [x] ~~Azure stage `shared` (plan + Log Analytics)~~
- [x] ~~Azure stage `prod` (web app + managed identity + federated credential)~~
- [x] ~~GitHub `prod` environment, Azure secrets, `PROD_DEPLOYERS`, `prod` ruleset~~
- [ ] First dev→prod PR merged and first Deploy prod run
- [ ] Scale plan to B1, bind groove.com.au + www, managed certificates, HTTPS Only

Per-release quick checklist (copy into each release PR):

- [ ] Dev lead approval in the PR
- [ ] Files changed reviewed; no unexpected workflow/infra edits
- [ ] `ci` green; dev site spot-checked
- [ ] Merged with a merge commit
- [ ] Deploy prod run on `prod` with a release label; both jobs green

- [ ] Live site and `/healthz` checked; release posted to team

Open items:

- Does the Cloudflare account host other Workers or Pages?
- GitHub team slugs for IT and the dev team
- GitHub logins for `PROD_DEPLOYERS`
- Confirm the org's GitHub plan is Team or higher
- Note: anyone who can push to `dev` can in effect read the Cloudflare token, so keep that token scoped to Workers only.

Command-line release (from the Mac)

The site doesn't go live from a git push alone. Git moves the code from `dev` to `prod`, and then the Deploy prod workflow publishes it. Here's the full sequence, run from your clone on the Mac.

1. Update local branches

```
git clone https://github.com/JRC-Software/groove-marketing-web.git
cd groove-marketing-web
git fetch origin
git checkout dev
git pull origin dev
git checkout prod
git pull origin prod
```

2. See what's going out

```
git log --oneline origin/prod..origin/dev
git diff --stat origin/prod origin/dev
```

Empty output means there's nothing new to release.

3. Merge dev into prod

Use the PR route. It runs the `ci` check and matches your release flow:

```
gh pr create --base prod --head dev --title "Release $(date +%Y-%m-%d)" --
body "Release dev to prod"
gh pr merge dev --merge
```

The pure-git alternative works only until `prod` is protected in P4, and it skips CI:

```
git checkout prod
git merge --no-ff origin/dev -m "Release $(date +%Y-%m-%d)"
git push origin prod
```

4. Confirm prod has everything from dev

```
git fetch origin
git log --oneline origin/prod..origin/dev
```

No output means `prod` contains all of `dev`.

5. Deploy (this is what makes it live)

```
gh workflow run deploy-prod.yml --ref prod -f release=$(date +%Y-%m-%d)
gh run watch $(gh run list --workflow deploy-prod.yml --limit 1 --json
databaseId --jq '.[0].databaseId')
```

6. Check the live site

```
curl -s https://prod-groove-marketingweb-appsvc-aue.azurewebsites.net/healthz
```

Compare the commit it reports with `git rev-parse --short origin/prod`.

Steps 1–4 work today. Step 5 will fail until P2–P4 are done: the Azure resources, the `prod` environment, its secrets, and your login in `PROD_DEPLOYERS`.